

README

Web-client LLM e2e — runtime evidence (HXC-103)

Revision	1
Created	2026-06-15
Last modified	2026-06-15
Status	active

Captured runtime evidence (§11.4.83) for the HelixCode web client's three LLM paths, driven end-to-end against a **live Ollama** provider (qwen2.5:3b at `http://localhost:11434`) — no stub, no mock, no canned response (CONST-050(A) / §11.4.107). Each PASS carries real captured output, not metadata.

Table of contents

- [Environment](#)
- [Path 1 — POST /api/v1/llm/generate \(HTTP\)](#)
- [Path 2 — POST /api/v1/llm/stream \(SSE\)](#)
- [Path 3 — browser → server → provider → browser](#)
- [Path 4 — POST /api/v1/specify \(speckit debate\)](#)
- [Production hang found + fixed](#)
- [Evidence files](#)

Environment

- Provider: live Ollama container `helix_ollama_video`, model `qwen2.5:3b`, on `localhost:11434` (a stale podman port-forward was re-established via `podman restart helix_ollama_video`).
- Tests: `tests/integration/*_e2e_test.go` (`//go:build integration`), each boots the real `server.New(...)` HTTP server on a free port and issues real client requests. SKIP-OK when the provider is unreachable (§11.4.3) — never a fake PASS.

Path 1 — POST /api/v1/llm/generate (HTTP)

`TestLLMGenerateE2E` — real `http.Client` POST → server → `resolveLLMProvider` (local Ollama) → real generation. **PASS**.

- HTTP 200 `body=map[content:4 finish_reason:stop model:qwen2.5:3b provider:ollama status:success usage:map[completion_tokens:2 prompt_tokens:42 total_tokens:44]]`
- Anti-bluff: the live model's real answer to "What is 2+2?" is "4" (arithmetic a canned response would not reliably solve). Raw provider round-trip also captured.

Path 2 — POST /api/v1/llm/stream (SSE)

TestLLMStreamE2E — real POST → server streamLLM → live Ollama streamed as Server-Sent Events. **PASS** (1.1s; deterministic over -count=3).

- 9 content SSE data: frames + terminal [DONE] sentinel.
- Aggregated streamed content: 1\n\n2\n\n3\n\n4\n\n5 (the live model's real count).
- Anti-bluff: > 1 **frame** is the load-bearing proof it genuinely streamed token-by-token (not a single buffered write); content contains "1" and "5".

Path 3 — browser → server → provider → browser

TestWebBrowserE2E_GenerateRoundTrip — a real headless Chrome (chromedp) loads the server-served frontend, types a prompt into the real textarea#prompt, triggers button#send, and reads the answer back out of the live DOM. **PASS** (2.7s).

- GET / 200, /static/app.css 200, /static/app.js 200 (frontend genuinely served).
- Rendered #output DOM text: "4" (the real model answer, read from the browser).
- Rendered #meta DOM text: provider=ollama model=qwen2.5:3b tokens=44 finish=stop.
- Screenshot captured during the run (chromedp CaptureScreenshot).

Path 4 — POST /api/v1/specify (speckit debate)

TestSpecifyServerE2E (HXC-105) — real POST → server specifyHandler → a 2-agent speckit debate driven by the live provider. The speckit path was previously proven only provider-direct (pillar.ExecutePhase); this proves it through the booted HTTP server. **PASS** (75.9s).

- HTTP 200, status:success, provider:ollama, model:qwen2.5:3b, qualityScore:0.9808.
- Real 3-round, 2-agent debate transcript (FOR/AGAINST/SYNTHESIS → CONCLUSION); metrics provider_calls=6 total_tokens=806 prove 6 genuine provider.Generate round-trips.
- Anti-bluff: non-empty output, NOT the synthesized "awaiting provider wiring" stub, provider == "ollama". Honest 502/503 (no fabricated output) when the provider is down.

Production hang found + fixed

The streaming e2e exposed a **real production defect** (HXC-104): streamLLM's goroutine called provider.GenerateStream(ctx, llmReq, chunkChan) but never closed chunkChan, so the SSE consumer never observed channel-drain, never emitted [DONE], and **every** /stream request hung until the 120s context deadline. Fix: defer close(chunkChan) in that goroutine (internal/server/llm_generate.go). This is exactly the class of user-facing breakage that handler/httpTest tests miss and a real runtime e2e catches.

Evidence files

- `raw_provider_generate.json` — raw Ollama `/api/chat` round-trip (answer "4").
- `generate_e2e_test.go` output: `generate_e2e_test_output.log`.
- `stream_and_browser_e2e_output.log` — the run that first exposed the hang/flake.
- `all_three_e2e_output.log` — all three paths PASS together post-fix.